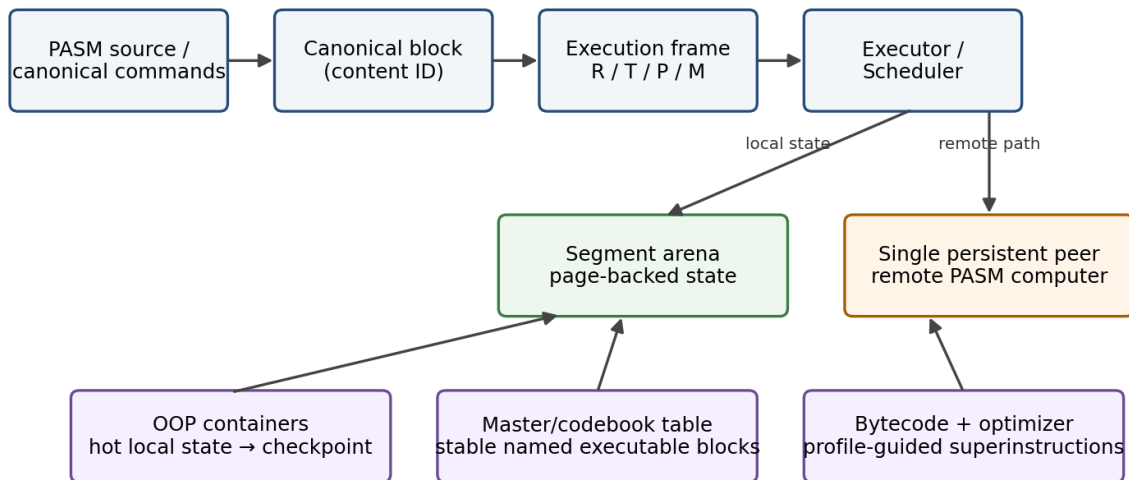


PASM

Programming Manual

Canonical Frames, Segmented Memory, High-Performance OOP, Bytecode, Compression, and Single-Peer Remote Execution

PASM: From Source to Local or Remote Execution



Architecture overview used throughout this manual.

Experimental Runtime Tutorial • 2026 Edition

A practical guide to programming the PASM runtime as it exists in the current project files.

STATUS: PASM is an experimental PHP-hosted virtual machine/runtime architecture. The manual documents implemented behavior and measured development benchmarks; it does not claim universal superiority over WebAssembly or native runtimes.

Preface

PASM began as a register-and-stack-oriented PHP instruction library. During the work documented in this project, it evolved into a broader runtime: canonical register frames, immutable content-addressed blocks, cooperative scheduling, segmented page-backed state, an optimized OOP container layer, compact bytecode, profile-guided superinstructions, deterministic codebook compression, and a persistent single-peer remote execution model. This manual teaches the current architecture as a programming system rather than as a collection of isolated PHP classes.

The central idea is separation. Code identity is separate from execution state. A canonical block says what work means; a frame says which instance is running it. Hot OOP operations are separate from checkpoint storage. Segments have stable logical handles even when physical pages move. Local execution is separate from remote transport, yet both use the same block and register identities. Compression is separate from prediction: PASM uses deterministic frequency tables and canonical Huffman coding rather than an AI model.

The goal of this book is not to hide the PHP implementation. You will see the PHP APIs, the canonical instruction arrays, the assembly/bytecode layer, and the network-facing model. Each chapter first establishes the mental model, then shows a working example, then explains what happens inside the runtime. Exercises are included so the reader can test whether the model is actually understood.

All benchmark tables are development measurements. They are useful for regression detection and engineering direction, not as universal performance guarantees. Where PASM has demonstrated an improvement, the manual says exactly what was measured; where PASM is still behind native PHP or has only an architectural hypothesis, the manual says that too.

How to Use This Book

- If you want to write local PASM programs first, read Parts I and II, then Chapters 9-13 on memory and containers.
- If you are primarily interested in remote execution, learn canonical blocks and frames first, then jump to Part V.
- If you are optimizing PASM itself, read the performance chapters and the appendices alongside the source files.
- If you are migrating older PASM code that uses static registers such as `ecx`, `ah`, and `rdx`, read Appendix F before rewriting everything.

CONVENTION: Canonical register examples use `R0`, `T0`, `P0`, and `M0`-style names. The older PASM core uses static registers such as `ecx`, `ah`, `adx`, and `rdx`. Both exist in the project, but the canonical model is the preferred architecture for new runtime work.

Table of Contents

Part I — Foundations

1. What PASM Is
2. Project Layout and Setup
3. Your First Canonical Program
4. Legacy PASM and the Canonical Transition

Part II — The Canonical Machine

5. Canonical Registers
6. Frames and Execution Identity
7. Canonical Blocks and Stable IDs
8. Control Flow, Return, and Yield
9. Cooperative Scheduling

Part III — Memory, Segmentation, and OOP

10. Segmented Page-Backed Memory
11. Defragmentation and Segmented Loops
12. The OOP Container Architecture
13. Vector, Stack, Queue, and Deque
14. Map and Set
15. Higher-Level Data Structures

Part IV — Bytecode, Fusion, and Compression

16. Compiled PASM Bytecode
17. Superinstructions and the Optimizer
18. Adaptive Profiling Without AI
19. Codebooks, Best-Bet Prediction, and Huffman Coding
20. The Persistent Master Execution Table

Part V — One Remote Computer

21. The Single-Peer Model
22. Remote Registers and Execution Frames
23. Remote Segments and Dirty-Page Sync
24. Network Safety and Failure Boundaries

Part VI — Projects and Performance

25. Project: A Reusable Remote Calculator
26. Project: Cooperative Work Pipeline
27. Project: Segmented Lookup Processor

- 28. Benchmarking PASM Correctly
- 29. Current Benchmark Results
- 30. Performance Tuning Checklist
- 31. What to Build Next

Part VII — Exercises and Guided Practice

Capstone Exercises

Selected Solution Hints

Appendices A-H — Reference, APIs, benchmarks, glossary, troubleshooting, migration, and design invariants

Contents at a Glance

Part I — Foundations: PASM’s purpose, setup, first program, and legacy/canonical relationship.

Part II — The Canonical Machine: registers, frames, blocks, execution, control flow, yield, and scheduling.

Part III — Memory and Objects: segments, defragmentation, containers, and data structures.

Part IV — Bytecode and Optimization: compilation, superinstructions, adaptive profiling, codebooks, and the master table.

Part V — One Remote Computer: persistent single-peer execution, remote frames, register windows, and segment synchronization.

Part VI — Projects and Performance: complete examples, benchmark methodology, tuning, and limitations.

Appendices — instruction reference, container API, data structures, file map, glossary, troubleshooting, and migration notes.

Part I — Foundations

Learn the programming model before optimizing it.

Chapter 1: What PASM Is

PASM is best understood as a PHP-hosted runtime with an assembly-like programming surface and a progressively more explicit machine model.

At the oldest layer, PASM is a class of static registers, flags, a stack, a chain of executed methods, and assembly-inspired methods. That layer is still useful for compatibility and for experimenting with instruction semantics. The newer architecture introduces a canonical machine because global static registers become awkward when the same work must run concurrently, yield, resume, or execute on another computer.

The canonical system therefore treats code and state as separate objects. Code is stored in immutable canonical blocks. State is stored in register frames and segment arenas. A scheduler moves execution between frames. A single-peer protocol can mirror selected state to one remote PASM executor. OOP containers use fast frame-local PHP representations while exposing canonical segmented checkpoints at synchronization boundaries.

PASM: From Source to Local or Remote Execution

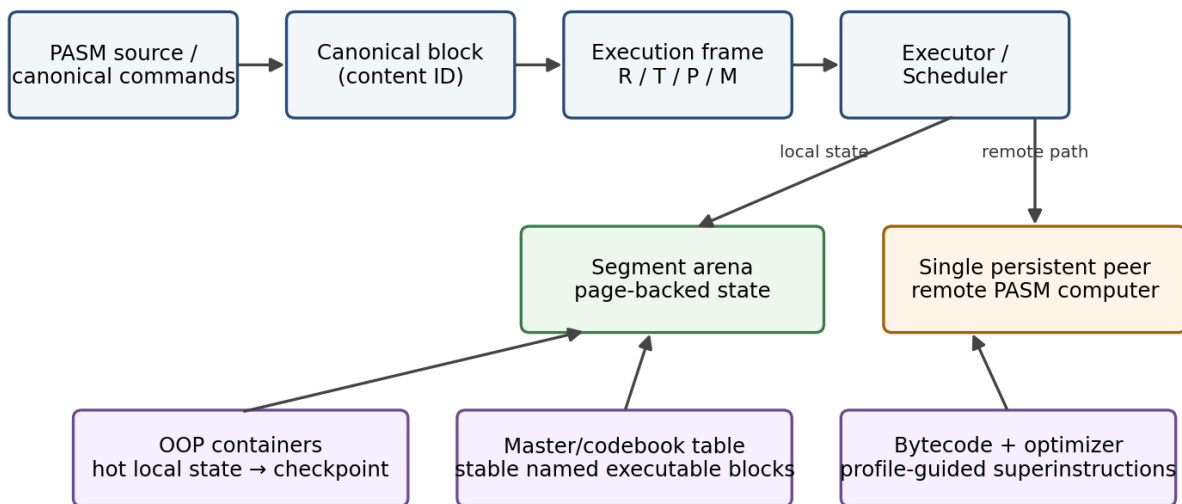


Figure 1.1 — PASM separates source, code identity, execution state, storage, optimization, and remote transport.

A useful way to think about PASM is as five cooperating layers:

- Instruction layer — arithmetic, bitwise operations, moves, comparisons, jumps, returns, yields, and segmented memory commands.
- State layer — canonical R/T/P/M registers plus frame-owned segments.
- Object layer — Vector, Stack, Queue, Deque, Map, Set, and higher-level data structures.
- Optimization layer — binary bytecode, superinstructions, adaptive profiling, deterministic codebooks, and persistent executable lookup tables.

- Peer layer — an authenticated persistent connection to one remote PASM executor using compact frame/register/block operations.

WHAT PASM IS NOT: PASM is not currently a general replacement for native machine code, WebAssembly, or a production operating-system scheduler. Its strongest demonstrated ideas are in runtime organization, compact repeated remote execution, optimized PHP-side containers, and specialized interpretation.

The project uses “canonical” to mean that a piece of machine state or executable work has a stable representation. For example, a canonical block ID is derived from its command content and schema rather than from a user-friendly display name. This allows the same work to be recognized on both sides of a connection without sending its source again.

The most important programming habit is to decide which layer you are using. If you are writing a simple local calculation, canonical blocks and frames may be enough. If you need a mutable collection of mixed PHP values, use the OOP container layer. If the program is repeated remotely, consider bytecode and the persistent block/codebook layers. Avoid mixing all layers into every operation merely because they exist.

Exercises

1. Explain in your own words why a canonical block and a register frame are separate objects.
2. List one situation where the legacy static-register model is simpler and one where a canonical frame is safer.
3. Identify which PASM layer should own a long-lived queue used by a cooperative task.

Chapter 2: Project Layout and Setup

A PASM program is ordinary PHP that loads one or more runtime layers and then defines executable work.

For the examples in this manual, place the PASM files in one directory and run them with PHP 8.4 or a compatible modern PHP build. CLI OPcache is useful when benchmarking, but it is not required to understand the runtime. The project has grown in layers, so the file you require depends on what you are using.

File	Purpose
pasm-double-digit.php	Optimized legacy PASM core.
pasm-canonical.php	Canonical registers, frames, blocks, executor, scheduler.
pasm-canonical-segmented.php	Canonical executor plus frame-owned segments/pages.
pasm-oop-containers.php	Hot-path canonical OOP containers.
pasm-data-structures.php	Linked lists, BST, heap, graph, trie, hash table, disjoint set.
pasm-bytecode.php / pasm-bytecode-optimized.php	Binary bytecode compiler/VM and optimized superinstructions.
pasm-codebook.php	Dictionary, best-bet predictor, Huffman transport.
pasm-canonical-peer.php	Persistent single-peer register/block execution.
pasm-segmented-peer.php	Single-peer segmented synchronization.

A minimal canonical program needs only ``pasm-canonical.php``. The canonical namespace is ``pasm``, so examples commonly import the classes they use with a PHP ``use`` statement.

```
<?php declare(strict_types=1);
require_once __DIR__ . '/pasm-canonical.php';

use pasm\{PASMCanonicalTable, PASMCanonicalExecutor, PASMFramePool};

$table = new PASMCanonicalTable();
$frames = new PASMFramePool();
$exec = new PASMCanonicalExecutor($table);
```

Use ``strict_types=1`` in tutorial code because the canonical register file is intentionally integer-oriented. Mixed PHP values belong naturally in OOP containers and are encoded only when a canonical checkpoint is built.

When benchmarking, run the same script multiple times and prefer medians. One useful command is:

```
php -d opcache.enable_cli=1 benchmark-pasm-oop-fast.php
```

REPRODUCIBILITY: Do not compare one PASM number from one machine to a number reported for a different runtime on another machine. Compare versions on the same machine,

with the same PHP build, workload, warm-up, and iteration count.

Exercises

1. Create a directory containing only `pasm-canonical.php` and a new `demo.php`. Make `demo.php` load without warnings.
2. Run ``php -v`` and record the PHP version you will use for your own benchmarks.
3. Explain why the OOP container file requires the segmented canonical layer even though hot operations do not touch segments every time.

Chapter 3: Your First Canonical Program

The smallest useful PASM program defines a block, creates a frame, loads inputs, invokes the block, and reads the result.

The following program adds two values. The block contains only the algorithm. The frame contains the values for one invocation.

```
<?php declare(strict_types=1);
require_once __DIR__ . '/pasm-canonical.php';

use pasm\{PASMCanonicalTable, PASMCanonicalExecutor, PASMFramePool};

$table = new PASMCanonicalTable();
$block = $table->define('add-two', [
    ['ADD', 'R2', 'R0', 'R1'],
    ['RET', 'R2'],
]);

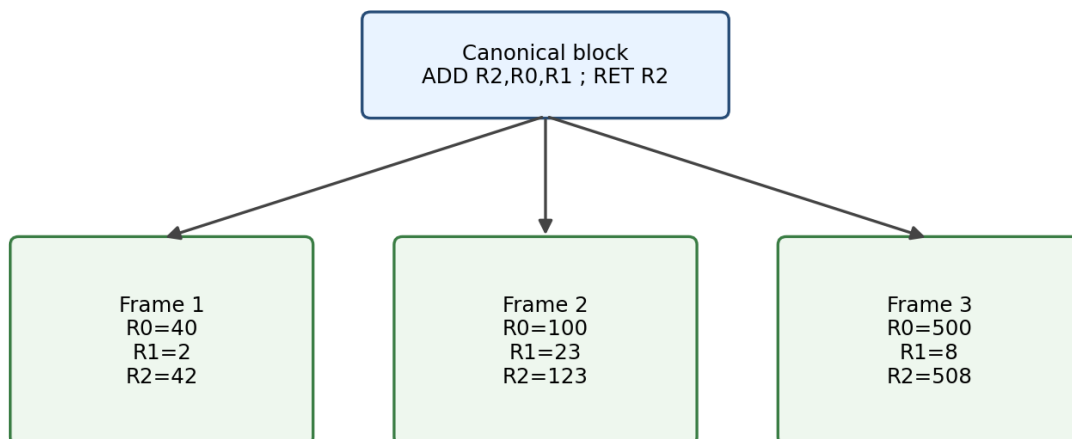
$frames = new PASMFramePool();
$frame = $frames->create('example');
$frame->set('R0', 40);
$frame->set('R1', 2);

$exec = new PASMCanonicalExecutor($table);
$result = $exec->invoke($block->id, $frame);

echo $result['result'];    // 42
```

The instruction `ADD R2, R0, R1` reads R0 and R1 and writes their sum into R2. `RET R2` marks the block complete and returns R2. The executor also copies the return value to P0. This convention makes P0 useful as a persistent/result register when a block is part of a larger sequence.

One Immutable Block, Many Independent Frames



Block ID identifies the code. Frame ID identifies one execution instance.

Figure 3.1 — One immutable add block can run against many independent frames.

Now run the same block twice without redefining it:

```
$a = $frames->create('a');
$a->set('R0', 40);
$a->set('R1', 2);

$b = $frames->create('b');
$b->set('R0', 100);
$b->set('R1', 23);

$ra = $exec->invoke($block->id, $a);
$rb = $exec->invoke($block->id, $b);

// 42 and 123; the code is shared, the state is not.
```

This block/frame separation is one of the most important PASM concepts. It lets a remote machine store a block once and accept many lightweight execution frames. It also lets the cooperative scheduler run the same block for many tasks without cloning the code.

DEBUGGING RULE: If two invocations unexpectedly affect each other, inspect the frame ownership first. Shared immutable blocks are expected; shared mutable register or segment state usually is not.

Exercises

1. Write a `multiply-two` block using MUL and verify $6 \times 7 = 42$.
2. Create three frames for one block and prove that changing R0 in one frame does not change the other two.
3. Change the result register from R2 to P3. What does RET return, and what remains in P0 afterward?

Chapter 4: Legacy PASM and the Canonical Transition

The legacy core remains a useful compatibility layer, but new concurrent and remote designs should avoid making global static registers the center of state.

The original PASM core exposes registers such as ``ecx``, ``ah``, ``adx``, ``bdx``, ``cdx``, ``ddx``, ``edx``, and ``rdx`` as static properties. It also keeps a static stack, flags, and an execution chain. That model is concise for small assembly-like sequences. The optimized ``pasm-double-digit.php`` improved that path by removing unnecessary method-name splitting and debug overhead, but static global state remains intrinsically difficult to isolate across tasks.

Canonical PASM solves that by replacing “the register” with “a register in a frame.” This does not mean the legacy core must disappear. The new OOP containers still provide explicit bridges to legacy PASM registers and the legacy stack for compatibility. The key is to use those bridges at boundaries, not on every hot container operation.

```
use pasm\Vector;

$v = new Vector([10, 20, 30]);
$v->loadPasmStack(replace: true);
// Legacy PASM::$stack now receives the values.

$v->loadRegister('P0');
// In the canonical frame, P0 receives the container ID.
```

That example shows two different meanings of “load register.” A canonical R/T/P/M register stores a compact integer identity. A legacy static register can still receive a PHP array representation. The canonical form is appropriate for machine identity and synchronization; the legacy form is convenient for compatibility with older code.

MIGRATION PRINCIPLE: Do not rewrite legacy code solely to rename registers. Rewrite when you need frame isolation, yield/resume, segmented storage, compact remote execution, or a canonical identity that persists across processes.

Exercises

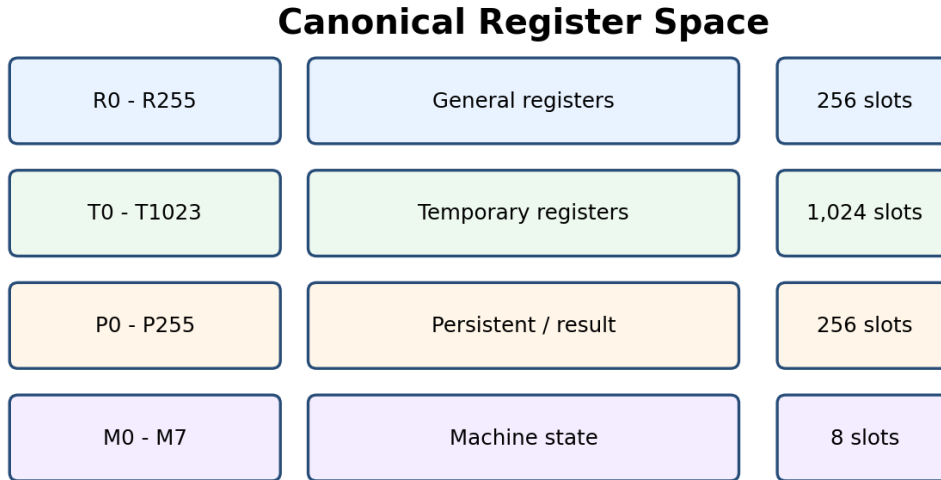
1. List the legacy registers you currently depend on and classify them as input, output, temporary, or machine state.
2. Design a mapping from a legacy calculation using `ecx/ah/rdx` into `R0/R1/P0`.
3. Explain why moving container data into a legacy global register on every push would defeat the hot-path OOP design.

Part II — The Canonical Machine

Registers, frames, immutable blocks, control flow, and cooperative tasks.

Chapter 5: Canonical Registers

PASM exposes a large sparse virtual register space without allocating one PHP property per register.



Sparse storage: an unused zero register does not require a dedicated PHP property.

Figure 5.1 — Four canonical register banks serve different lifetimes and roles.

Every execution frame exposes 1,544 addressable canonical registers: 256 general R registers, 1,024 temporary T registers, 256 persistent/result P registers, and 8 machine M registers. The implementation stores only nonzero entries, so an untouched register does not consume an individual PHP property.

Bank	Range	Recommended role
R	R0-R255	Inputs, ordinary working values, arguments.
T	T0-T1023	Compiler/runtime temporaries; clearable scratch state.
P	P0-P255	Persistent values, handles, results, segment/container IDs.
M	M0-M7	Instruction pointer, flags, frame/block/status and other machine state.

Register names are converted to stable numeric IDs by ``PASMCanonicalISA``. Numeric IDs matter on the wire because a register can be represented compactly rather than transmitting a string such as ``R255`` repeatedly.

```
$frame->set('R0', 40);  
$frame->set('T1023', 9);  
$frame->set('P255', 123);  
  
echo $frame->get('R0');
```

A canonical frame currently stores integer values. This is intentional. Strings, floats, arrays, and objects are better represented inside the OOP container layer, which uses a frame-local value pool when it builds a canonical checkpoint image. Treat the canonical registers as machine words and handles, not as unrestricted PHP variables.

REGISTER DISCIPLINE: Choose a register contract for each canonical block. For example: R0/R1 inputs, T0-T3 scratch, P0 return value, P1 segment handle. Put the contract in the block schema when the block will be reused widely.

Exercises

1. Create a frame and set R255, T1023, and P255. Verify all three values.
2. Design a register convention for a block that receives three integers, uses four temporaries, and returns two persistent outputs.
3. Why is a sparse register array preferable to 1,544 PHP object properties per frame?

Chapter 6: Frames and Execution Identity

A frame is the mutable state of one execution instance.

`PASMFramePool` creates, retrieves, and drops frames. A frame has a numeric ID and a human-readable name. The `M_FRAME` machine register is initialized with the frame ID. The pool allows the scheduler and peer code to refer to frames compactly.

```
$frames = new pasm\PASMFramePool();

$frameA = $frames->create('request-A');
$frameB = $frames->create('request-B');

$frameA->set('R0', 40);
$frameB->set('R0', 100);

// Same register name, different execution state.
```

Frame isolation is more than a convenience. It makes concurrency and remote execution comprehensible. A block can be cached once because it is immutable, while each active task or remote request carries only its frame-specific state. The same idea extends to segmentation: each frame can own a segment with logical ID 1 without colliding with another frame's segment 1.

Temporary registers can be cleared without destroying persistent results:

```
$frame->set('T0', 77);
$frame->set('P0', 42);
$frame->clearTemps();

// T0 -> 0, P0 -> 42
```

Snapshots are useful for diagnostics and persistence of a local frame:

```
$snapshot = $frame->snapshot();
$copy = pasm\PASMRegisterFrame::fromArray($snapshot);
```

LIFETIME MODEL: Blocks are long-lived and shareable. Frames are execution instances. Remote frames are session state and are intentionally recreated after reconnect, while canonical block tables can persist across restarts.

Exercises

1. Create two frames with the same name but different IDs. Explain why the ID, not the name, is the machine identity.
2. Snapshot a frame, restore it, and verify R/T/P values.
3. What state should survive a remote node restart: block IDs, frames, or both? Defend your answer.

Chapter 7: Canonical Blocks and Stable IDs

A canonical block is immutable executable content with a deterministic identity.

``PASMCanonicalTable::define()`` creates a block from a display name, an ordered command array, and an optional schema. The canonical ID is derived from executable content. Two blocks with identical commands and schema can therefore represent the same machine work even if humans use different aliases.

```
$block = $table->define('math.add', [
    ['ADD', 'R2', 'R0', 'R1'],
    ['RET', 'R2'],
], [
    'inputs' => ['R0', 'R1'],
    'output' => 'R2',
]);

echo $block->id;    // 24-hex content identity
```

The canonical table may be constructed with a persistence file. When persistence is enabled, defining or installing a block writes the table atomically. On a remote executor, this allows the node to restart and still recognize a previously installed block ID.

```
$table = new pasm\PASMCanonicalTable('/var/lib/pasm/master.json');
$table->define('math.add', $commands, $schema);
```

Treat aliases as UI and IDs as protocol. A human can rename ``math.add`` to ``arithmetic.add`` without changing what the machine considers the executable content, as long as the commands and schema are unchanged.

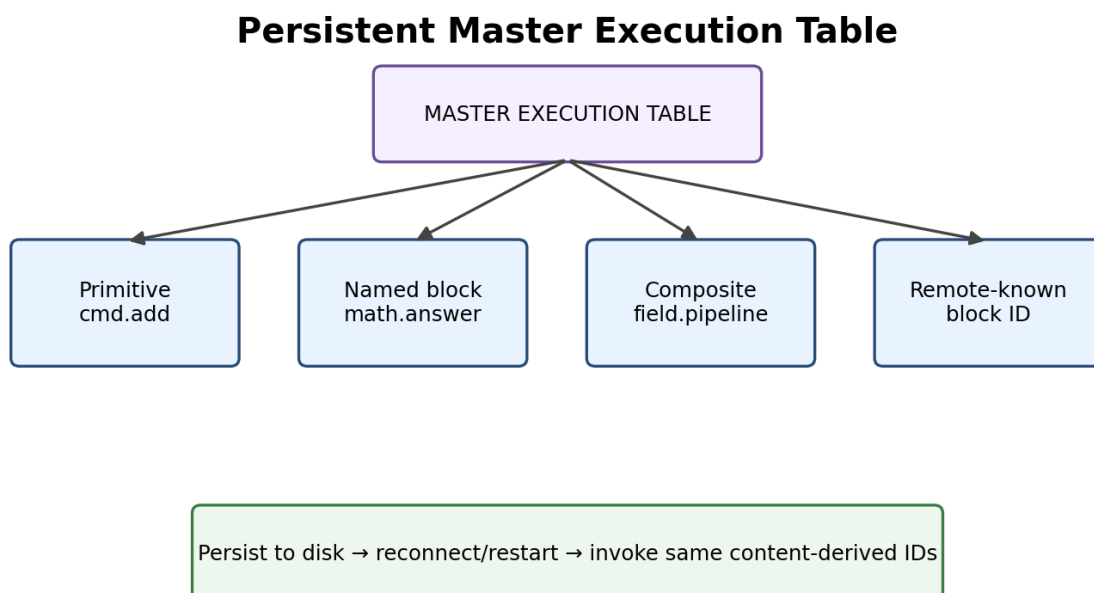


Figure 7.1 — The persistent execution vocabulary can contain primitives, blocks, composites, and remotely known IDs.

Exercises

1. Define two aliases for identical command arrays and compare their content IDs.
2. Change one command operand and observe that the content ID changes.
3. Write a schema for a block that owns P1 as a segment handle and returns P0.

Chapter 8: Control Flow, Return, and Yield

Canonical PASM provides a small explicit control-flow core that is easy to inspect and constrain.

The canonical executor supports `MOVI`, `MOV`, arithmetic, bitwise operations, `INC/DEC`, `CMP`, `JMP/JZ/JNZ`, `RET`, `YIELD`, and `HALT`. Jumps use command indexes in canonical command arrays. ``CMP`` records equality in the machine flags register and in the executor's current comparison state; ``JZ`` and ``JNZ`` branch accordingly.

```
$countdown = $table->define('countdown', [  
    ['MOVI', 'R1', 0],  
    // pc 1  
    ['DEC', 'R0'],  
    ['CMP', 'R0', 'R1'],  
    ['JNZ', 1],  
    ['RET', 'R0'],  
]);
```

Use a budget to force an executor to yield after a fixed number of instructions. The returned structure includes state, program counter, result, and step count. The caller can invoke the same block again with the returned program counter.

```
$r = $exec->invoke($block->id, $frame, startPc: 0, budget: 64);  
  
if ($r['state'] === 'yielded') {  
    $r = $exec->invoke($block->id, $frame, $r['pc'], 64);  
}
```

Explicit ``YIELD`` is different from budget-based yielding: the program itself declares a scheduling boundary. Both preserve frame state. ``RET`` writes the result to `P0` and marks the frame done. ``HALT`` stops without necessarily producing a new result.

LOOP SAFETY: When creating dynamic or remote programs, always enforce instruction limits. A backward jump is legitimate control flow, but an unintended infinite loop must not hold the remote executor forever.

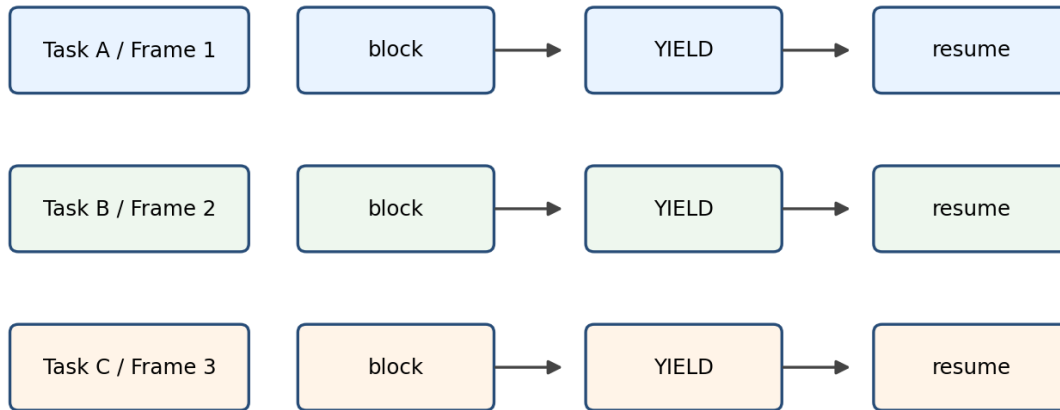
Exercises

1. Write a loop that increments `R0` from 0 to 10 and returns the final value.
2. Insert `YIELD` halfway through a block and manually resume it using the returned PC.
3. Compare `HALT` and `RET` for a block that is used only for side effects on persistent registers.

Chapter 9: Cooperative Scheduling

PASM schedules frames, not copied code.

Cooperative Scheduler: Yield and Resume



The scheduler rotates tasks; each task keeps its own frame and program counter.

Figure 9.1 — Round-robin scheduling moves execution ownership while frame state remains isolated.

`PASMFrameScheduler` is a cooperative round-robin scheduler. `spawn()` creates a frame, applies initial register values, and associates the frame with a block ID and program counter. `tick()` runs one ready task for the configured quantum. `run()` continues until no runnable tasks remain.

```
$scheduler = new pasm\PASMFrameScheduler($exec, $frames, quantum: 4);  
  
$a = $scheduler->spawn($block->id, ['R0' => 40, 'R1' => 2], 'task-42');  
$b = $scheduler->spawn($block->id, ['R0' => 100, 'R1' => 23], 'task-123');  
  
$events = $scheduler->run();
```

The block is shared; the frames are not. This is the mechanism that creates room to “yield and thread” without cloning every instruction stream. PASM’s current scheduler is cooperative, so it should be understood as lightweight task interleaving rather than true parallel CPU execution. True parallelism would require multiple PHP processes, fibers/threads, or remote executors.

A useful design pattern is to make a canonical block do one bounded unit of work and insert explicit YIELD points between major phases. That gives the scheduler predictable points at which to interleave other tasks and at which OOP containers can flush or remote segments can synchronize.

Exercises

1. Spawn three copies of one block with different R0 inputs and print the event sequence.

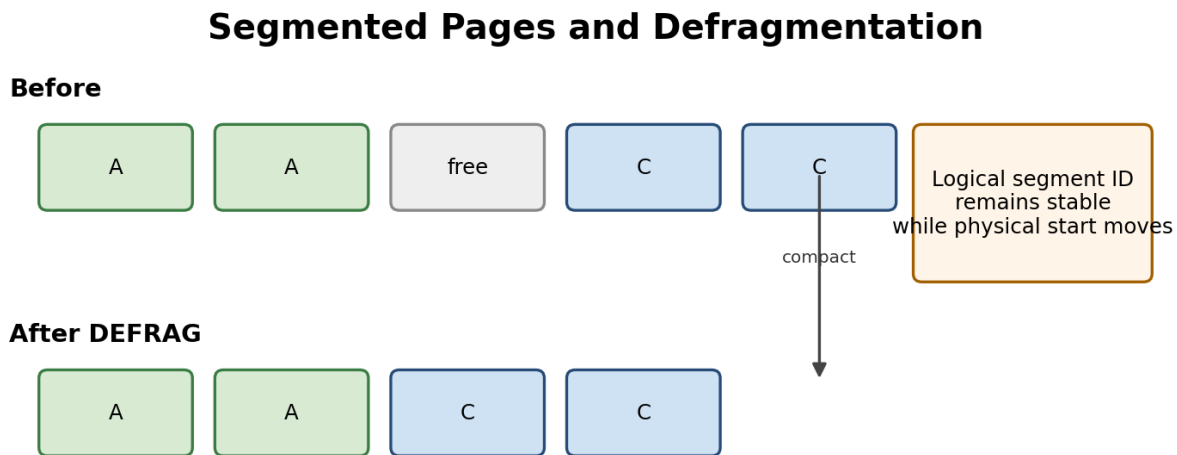
2. Change the scheduler quantum from 64 to 1 and describe the effect on event frequency and overhead.
3. Design a three-phase task that yields after read, transform, and write stages.

Part III — Memory, Segmentation, and OOP

Build state that can move, compact, yield, and synchronize without making every operation expensive.

Chapter 10: Segmented Page-Backed Memory

Segments give a frame stable logical handles to page-aligned integer storage.



Defragmentation compacts PASM logical pages; it does not claim to compact PHP's internal heap.

Figure 10.1 — Physical pages may move during compaction while the logical segment handle stays stable.

`PASMSegmentArena` allocates logical segments rounded to a page boundary. The default page size is 32 cells. A request for 80 logical cells therefore reserves three physical pages, or 96 PASM cells, while the logical length remains 80. Sparse zero cells do not need a stored value entry.

The segmented canonical executor adds commands such as SEGNEW, SEGFREE, SEGSET, SEGGET, SEGFILL, SEGCOPY, DEFRAG, and SEGSTAT.

```
$block = $stable->define('paged.work', [  
    ['SEGNEW', 'P1', 80, 'lookup'],  
    ['SEGSET', 'P1', 0, 'R0'],  
    ['SEGSET', 'P1', 33, 'R1'],  
    ['SEGGET', 'R2', 'P1', 0],  
    ['SEGGET', 'R3', 'P1', 33],  
    ['ADD', 'P0', 'R2', 'R3'],  
    ['RET', 'P0'],  
]);
```

P1 stores the logical segment ID. The physical start page can change later. That distinction is what makes defragmentation possible without rewriting every block that holds the handle.

The arena exposes statistics such as live segment count, used cells, high-water mark, fragmented cells, and free cells. These are runtime-level metrics, not claims about the PHP process heap.

MEMORY MODEL: PASM segmentation is a logical virtual memory layer implemented inside

PHP. DEFRAG compacts PASM segment placement; it cannot physically reorganize PHP's own allocator.

Exercises

1. Allocate an 80-cell segment with a 32-cell page size. Calculate its physical span.
2. Write values to offsets 0 and 33, then read them back through SEGGET.
3. Use SEGSTAT to observe the high-water mark before and after freeing a segment.

Chapter 11: Defragmentation and Segmented Loops

Compact segmented state at useful boundaries, not on every iteration.

A fragmented arena occurs when freed spans leave holes between live segments. `defrag()` compacts live segments toward page zero and returns information about moved and reclaimed cells. Because segment IDs are logical, programs continue to refer to the same handle after the physical start changes.

For segmented loops, the intended optimization is to compact once before the loop begins if fragmentation is present, then run all iterations against the stable packed layout. Defragmenting every iteration would convert a memory-maintenance operation into a hot-loop tax.

```
// Conceptual segmented loop boundary
DEFRAG
loop:
    SEGGET R2, P1, R1
    INC    R1
    CMP    R1, R3
    JNZ    loop
```

The project includes a segmented-loop auto-defrag test variant that detects backward branches whose loop bodies touch segmented commands and prepares the arena once per block invocation. If you use a build without that optional automatic layer, call `DEFRAG` explicitly before the segmented loop or at the higher-level scheduling boundary.

Measured on the included segmentation benchmark in the current environment, compacting an intentionally fragmented 1,000-page arena took about 0.834 ms and reduced the high-water mark from 31,968 to 16,000 cells, reclaiming 15,968 logical cells. A 100,000-pair segment write/read workload completed in about 37.666 ms, approximately 2.65 million pairs per second. Those are PHP implementation measurements and should be rerun on your machine.

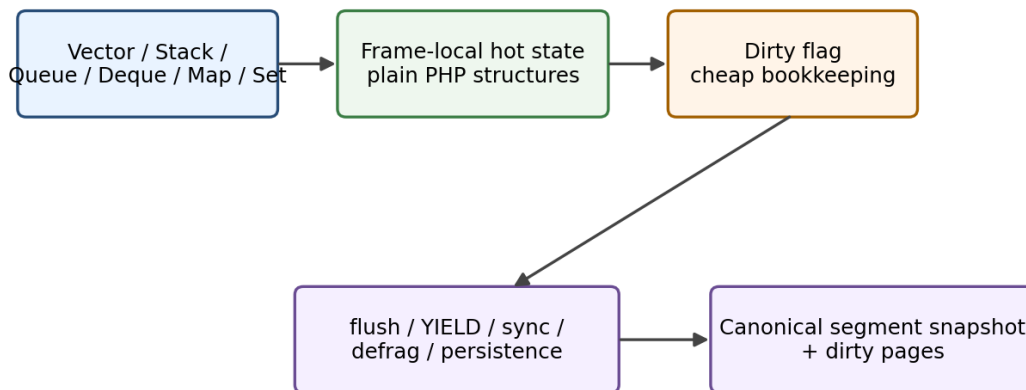
Exercises

1. Create A, B, and C segments; free B; record C's physical start; defrag; verify C's logical ID and contents are unchanged.
2. Explain why a segmented loop should normally defrag once before entry rather than once per iteration.
3. Design a heuristic that skips defrag when `fragmented_cells` is below a chosen threshold.

Chapter 12: The OOP Container Architecture

PASM containers keep the public OOP API fast by separating hot state from canonical checkpoint state.

OOP Containers: Hot Path vs Canonical Boundary



Normal add/get/push/pop does not encode each value into segmented cells.

Figure 12.1 — Hot methods remain local; canonical page materialization occurs only at explicit boundaries.

The first canonical OOP rewrite was correct but slow because each container operation encoded values and touched segmented storage. The current hot-path rewrite fixes that regression: normal methods use ordinary frame-local PHP structures, set a cheap changed flag, and defer cell encoding plus page import until a canonical boundary such as flush, dirtySegments, defrag, register export, remote synchronization, or persistence.

This architecture is crucial. Segmentation provides a transportable and compact checkpoint format, but it should not be in the inner loop of every push, pop, get, or put. The container knows when its logical contents changed; the checkpoint system knows how to compare the new image to the previous canonical image and mark only changed pages dirty.

```
use pasm\Vector;

$v = new Vector([10, 20, 30]);
$v->add(40);           // hot local state only
$answer = $v->get(3);  // hot local state only

$dirty = $v->dirtySegments(); // flush + compare checkpoint pages
$v->clearDirty();
```

Each container belongs to a canonical frame and owns a logical snapshot segment. `containerId()` identifies the object within the frame. `segmentIds()` returns the current backing segment IDs.

`loadRegister('P0')` stores the container identity in a canonical register so other PASM-aware code in the same frame can look it up.

Dirty-Page Synchronization



A canonical boundary materializes the logical state. Only pages whose encoded image changed need export.

Figure 12.2 — At a boundary, only pages whose canonical image changed need synchronization.

Exercises

1. Create a Vector, add 1,000 values, and confirm that no explicit flush is needed to read them locally.
2. Call `dirtySegments()`, `clearDirty()`, mutate one element, and call `dirtySegments()` again. Compare the reported pages.
3. Explain why a changed flag is cheaper than encoding every value after every method call.

Chapter 13: Vector, Stack, Queue, and Deque

The linear containers preserve familiar OOP interfaces while using specialized hot representations.

`PASMList`/`Vector` uses a packed PHP array. This is a good hot representation for append and indexed get/set. Insert and remove in the middle still use array splicing and therefore have the expected shifting cost; PASM does not pretend a vector has linked-list complexity.

```
use pasm\Vector;

$v = new Vector([10, 20, 30]);
$v->add(40);
$v[1] = 25;
echo $v->first(); // 10
echo $v->last();  // 40
```

`PASMStack` inherits the packed list representation and adds push, pop, and peek. The hot path is direct append/pop behavior.

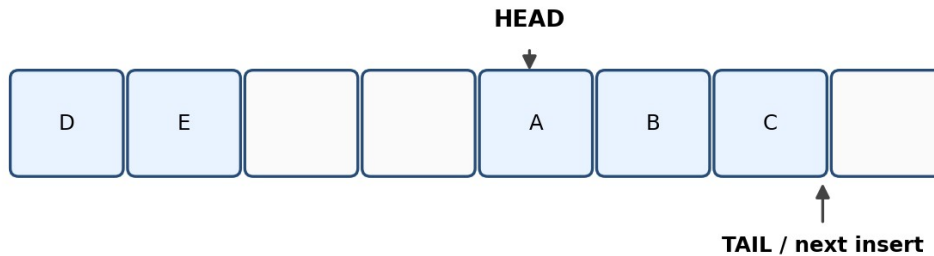
```
$s = new pasm\Stack();
$s->push('A')->push('B');
echo $s->peek(); // B
echo $s->pop();  // B
```

`PASMQueue` uses append plus a head index. Dequeue advances the head rather than shifting the entire array on every operation. Periodic compaction can reclaim consumed prefix storage when worthwhile.

```
$q = new pasm\Queue();
$q->enqueue(1)->enqueue(2)->enqueue(3);
echo $q->dequeue(); // 1
```

`PASMDeque` uses a power-of-two circular ring. That change fixed the worst path in the legacy implementation, which rebuilt arrays for opposite-end operations.

Circular Deque



Indexing uses a power-of-two mask; pushFront, pushBack, popFront, and popBack are $O(1)$ amortized.

Figure 13.1 — Circular indexing keeps all four deque-end operations $O(1)$ amortized.

```
$d = new pasm\Deque();  
$d->pushFront('B')  
->pushFront('A')  
->pushBack('C');  
  
echo $d->popBack(); // C  
echo $d->popFront(); // A
```

In the current development benchmark, 20,000 legacy `pushFront`/`popBack` operations took roughly 5.31 seconds, while the circular deque took about 1.81 ms. That dramatic result applies to a previously pathological implementation path; it should not be generalized into a 2,900× claim for all PASM operations.

Exercises

1. Use Vector as an array with [] syntax and verify ArrayAccess behavior.
2. Implement a browser-history example with Stack.
3. Use Queue to process ten jobs FIFO.
4. Use Deque to build a sliding-window buffer that can evict from either end.

Chapter 14: Map and Set

Map and Set use native PHP hashing on the hot path while retaining canonical checkpoint behavior.

`PASMMMap` stores `int|string` keys in a PHP associative array during ordinary execution. This is the correct hot representation in PHP. At a canonical boundary, the map builds a stable key/value checkpoint image for its segment. The API includes `put`, `get`, `has`, `remove`, `keys`, `values`, `ArrayAccess`, `iteration`, and `JSON` serialization.

```
$m = new pasm\Map();
$m->put('answer', 42)
    ->put('name', 'PASM');

echo $m->get('answer');
$m['mode'] = 'canonical';
```

`PASMSet` keeps a typed signature index. Scalars take a fast deterministic path; complex values use a fallback representation only when necessary. This removed the old behavior of serializing every value simply to test membership.

```
$a = new pasm\Set([1, 2, 3]);
$b = new pasm\Set([3, 4, 5]);

$u = $a->union($b);      // 1,2,3,4,5
$i = $a->intersect($b);   // 3
$d = $a->difference($b);  // 1,2
```

The Set rewrite is a good example of using the benchmark as a design tool rather than a scoreboard. The legacy Set's membership path paid serialization cost for ordinary scalars. The typed-signature rewrite removed that unnecessary work and improved the measured one-million-operation workload from about 240.272 ms to 152.258 ms in the current recorded benchmark.

Exercises

1. Use Map to count occurrences of words in a small list.
2. Use Set to remove duplicates from an input list while preserving the set's iteration behavior.
3. Compare Set membership behavior for integers and arrays and explain why the complex path is expected to cost more.

Chapter 15: Higher-Level Data Structures

PASM includes standard data structures for algorithms that are clearer than forcing everything into a single container type.

The data-structure layer includes singly and doubly linked lists, a binary search tree, min/max heap, graph, trie, hash table, and disjoint-set/union-find. These structures predate some of the latest canonical OOP work, so use them primarily for algorithmic behavior and bridge results into canonical frames at explicit boundaries when needed.

Structure	Typical use	Key methods
SinglyLinkedList	Sequential append/prepend/shift	append, prepend, shift, first, last
DoublyLinkedList	Two-ended linked sequence	pushFront, pushBack, popFront, popBack
BinarySearchTree	Ordered key lookup	put, get, has, min, max
BinaryHeap	Priority queue	minHeap/maxHeap, push, peek, pop
Graph	Traversal / adjacency	addEdge, neighbors, bfs, dfs
Trie	Prefix lookup	put, has, get, prefix
HashTable	Explicit hash storage	put, get, has, remove
DisjointSet	Connectivity / components	makeSet, find, union, connected

```
$heap = pasm\BinaryHeap::minHeap([8, 3, 12, 1]);  
echo $heap->pop(); // 1  
  
$graph = new pasm\Graph();  
$graph->addEdge('A', 'B')->addEdge('A', 'C')->addEdge('B', 'D');  
print_r($graph->bfs('A')); // A, B, C, D  
  
$trie = new pasm\Trie();  
$trie->put('register', 1)->put('reset', 2)->put('return', 3);  
print_r($trie->prefix('re'));
```

Choose the structure from the access pattern, not from how “low level” it sounds. A heap is appropriate for priority scheduling; a queue is better for FIFO work; a graph is clearer for relationships; a trie is clearer for prefix matching. PASM’s canonical machine model does not eliminate normal algorithmic data-structure decisions.

Exercises

1. Build a min-heap of five priorities and pop them in sorted order.
2. Create a graph with six vertices and compare BFS and DFS traversal order.
3. Use DisjointSet to model two initially separate groups and then union them.

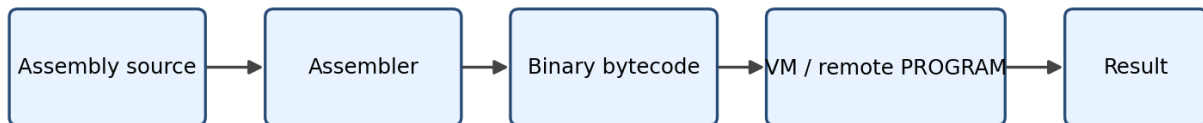
Part IV — Bytecode, Fusion, and Compression

Move from command arrays to compact executable representations and deterministic prediction.

Chapter 16: Compiled PASM Bytecode

Binary bytecode reduces parsing work and allows an entire remote program to travel in one request.

Compiled Bytecode Pipeline



Compile once, reuse many times. Remote execution sends the program once, then returns only the result.

Figure 16.1 — PASM assembly can be compiled once and executed repeatedly or transmitted as one PROGRAM payload.

The bytecode layer compiles assembly text into a compact binary instruction stream. The base instruction set includes HALT, MOVI/MOVR, arithmetic, bitwise operations, CMP, jumps, PUSH/POP, LOAD32/STORE32, INC/DEC/NEG, and RET. The bytecode VM maps its compact register IDs onto the legacy PASM register set used by that layer.

```
MOVI ecx 40
MOVI ah 2
ADD adx ecx ah
RET adx

$runtime = new pasm\PASMRuntime();
$assembler = new pasm\PASMOptimizingAssembler();
$vm = new pasm\PASMOptimizedBytecodeVM($runtime);

$code = $assembler->compile($source);
$result = $vm->run($code);
```

In the current local rerun used for this manual, compiling the four-instruction example 10,000 times took about 36.080 ms, while executing 10,000 precompiled programs took about 10.585 ms, around 944,702 executions per second. The compiled program was 26 bytes before superinstruction fusion. The important lesson is not the exact number; it is the value of compiling once and reusing the bytecode.

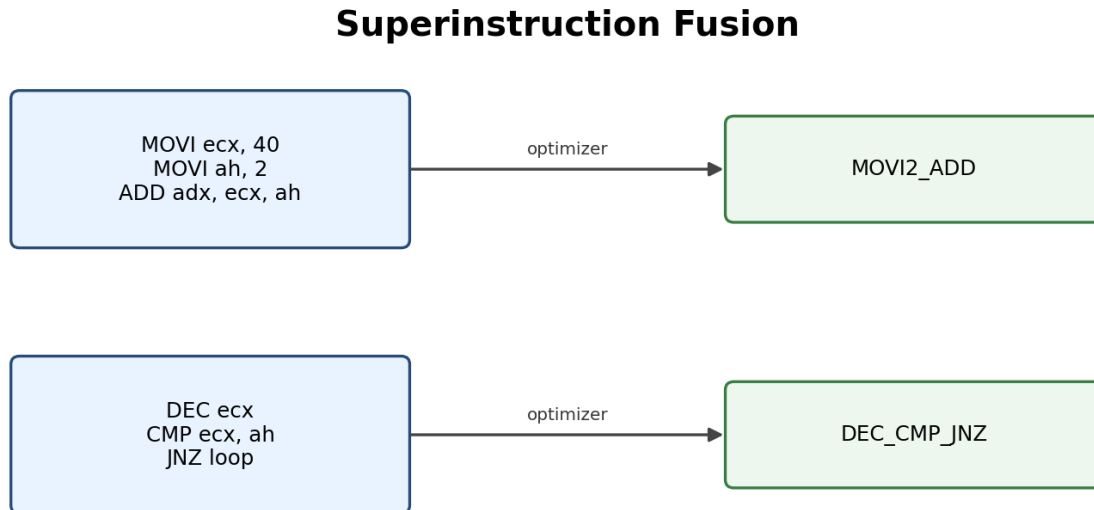
REMOTE ADVANTAGE: Sending one bytecode program and receiving one result avoids a network round trip per instruction. This is a plausible PASM advantage in network-bound workloads even when a native/JIT runtime has higher raw arithmetic throughput.

Exercises

1. Write bytecode that calculates $(7 \times 6) + 1$.
2. Compile one program once, execute it 100,000 times, and compare with recompiling every iteration.
3. Explain why program-size reduction matters more on a remote link than in a one-shot local calculation.

Chapter 17: Superinstructions and the Optimizer

Fusion reduces interpreter dispatch by replacing common multi-instruction patterns with vetted compound operations.



Fewer VM dispatches can improve interpreter throughput while preserving semantics.

Figure 17.1 — Safe recurring patterns can be represented by one fused opcode.

The optimizer currently recognizes safe templates such as `MOVI+MOVI+ADD` → `MOVI2_ADD`, `MOVI+MOVI+MUL` → `MOVI2_MUL`, `CMP+JZ/JNZ`, `DEC+CMP+JNZ`, and `LOAD32+ADD`. It does not fuse across labels, and it recomputes branch offsets after optimization.

In the manual’s current rerun at 20,000 executions per case, constant `ADD` improved from about 20.854 ms to 18.681 ms (1.12×), constant `MUL` from 23.268 ms to 18.613 ms (1.25×), and a 25-iteration `DEC/CMP/JNZ` loop from 333.339 ms to 200.541 ms (1.66×). Program sizes also shrank slightly: 26→22 bytes for the constant arithmetic cases and 32→29 bytes for the loop.

These numbers are PASM-vs-PASM measurements. They demonstrate that interpreter dispatch matters and that fusion can help. They do not establish a general advantage over WebAssembly, native PHP operations, or a JIT compiler.

Use superinstructions when a repeated sequence is common enough that one fused semantic handler is cheaper than multiple interpreter dispatches. Avoid creating hundreds of rarely used opcodes merely because a profiler observed them once; a large instruction vocabulary can increase implementation complexity and reduce maintainability.

Exercises

1. Find a three-instruction sequence in one of your programs that could plausibly be fused.

2. Explain why an optimizer must recalculate jump offsets after changing instruction sizes.
3. Benchmark a loop before and after fusion and report both time and bytecode size.

Chapter 18: Adaptive Profiling Without AI

PASM can learn hot instruction patterns using counters and n-grams without using a neural model.

The adaptive layer records actual executed opcode traces and counts recurring sequences. During warm-up runs, `PASMTraceProfile` records 2-4 instruction n-grams. `PASMProfileGuidedAssembler` then enables only known-safe fusion templates whose measured hit counts pass a threshold. `PASMAaptiveProgram` can automatically tier from profiling mode to optimized mode after a configured number of warm-up runs.

```
$program = new pasm\PASMAaptiveProgram(  
    new pasm\PASMRuntime(),  
    $source,  
    warmupRuns: 3,  
    minHits: 2  
);  
  
for ($i = 0; $i < 10; $i++) {  
    $result = $program->run();  
}  
  
print_r($program->plan());
```

The “intelligence” is explicit statistics. If `DEC>CMP>JNZ` appears 300 times and `MOVI>MOVI>ADD` appears 3 times, the profile says so. A deterministic policy decides which registered fusion templates are eligible. This is closer to profile-guided optimization or a small trace JIT strategy than to AI.

BEST-BET PHILOSOPHY: PASM’s adaptive systems use observed counts, stable rules, and measurable thresholds. They can be inspected, reproduced, reset, saved, and compared.

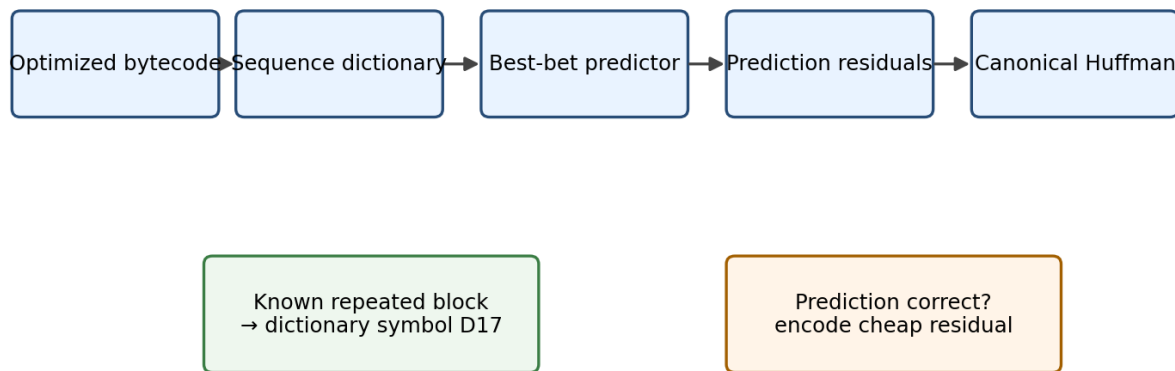
Exercises

1. Profile a loop and print the top five instruction n-grams.
2. Change `minHits` and observe which fusions remain enabled.
3. Describe one risk of generating arbitrary new PHP handlers directly from an execution trace.

Chapter 19: Codebooks, Best-Bet Prediction, and Huffman Coding

PASM compresses repeated executable structure using deterministic dictionaries and context statistics.

Codebook + Best-Bet + Huffman Transport



No neural model: deterministic frequency counts and canonical entropy coding.

Figure 19.1 — Repeated instruction sequences become dictionary symbols; best-bet prediction reduces the residual stream; canonical Huffman coding encodes what remains.

The codebook layer looks for exact recurring bytecode sequences, turns them into dictionary symbols, builds an order-2 transition table, and records the most frequent next token for each context. A correct prediction can be represented by a cheap residual symbol. Canonical Huffman coding then assigns shorter codes to more frequent residuals. The receiver reconstructs the exact original bytecode and validates CRC32 before execution.

```
$model = (new pasm\PASMBestBetModel())->observe([
    'LOAD32', 'ADD', 'STORE32',
    'LOAD32', 'ADD', 'STORE32',
]);

$bet = $model->bestBet(['LOAD32', 'ADD']);
// symbol = STORE32, confidence = 1.0 in this tiny example
```

Compression is not forced. The transport compares raw and codebook representations and sends raw bytecode when dictionary overhead would make a small program larger. This simple rule is important: an optimization should be allowed to decline work.

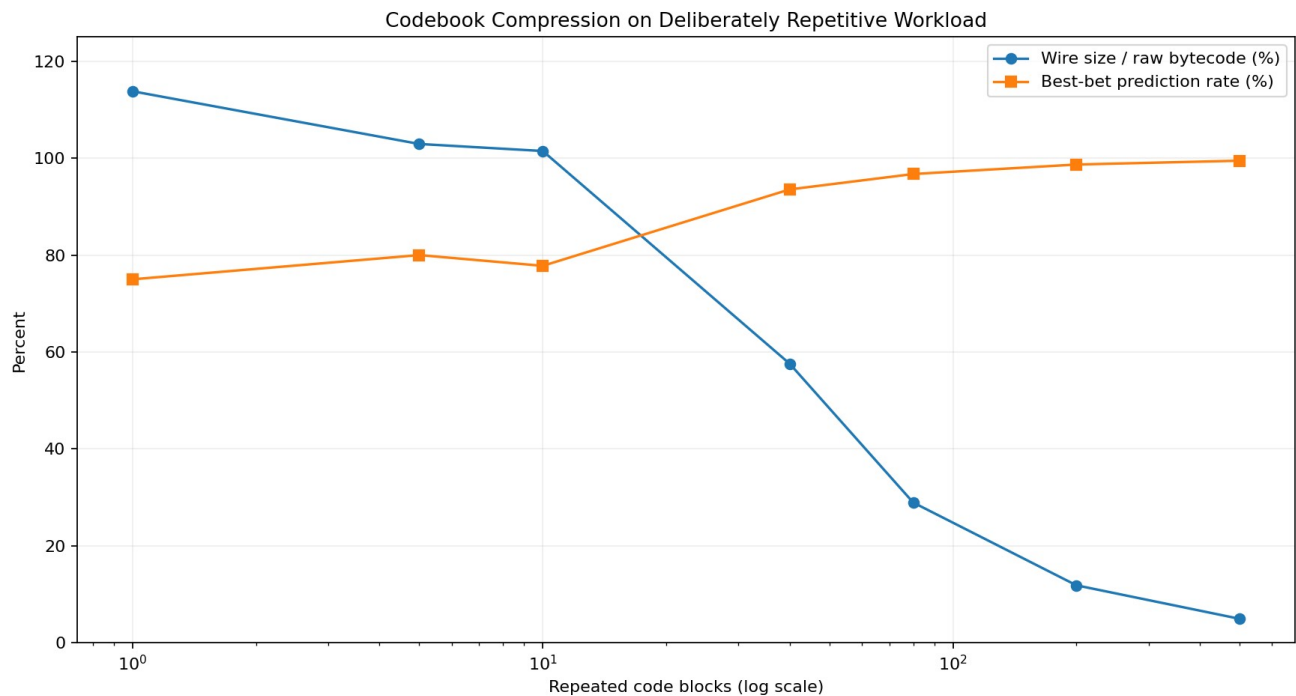


Figure 19.2 — Current codebook benchmark on an intentionally repetitive workload.

In the current rerun, 40 repeated blocks compressed from 1,082 raw bytes to 623 wire bytes; 80 blocks from 2,162 to 626; 200 from 5,402 to 641; and 500 from 13,502 to 670 bytes. Best-bet prediction rates increased as the deliberately repetitive stream grew. These ratios are not universal; irregular programs will compress differently, and tiny programs used the raw fallback.

Exercises

1. Build a best-bet model for a sequence of your own and inspect confidence for a two-token context.
2. Explain why dictionary overhead can make a 29-byte program larger rather than smaller.
3. Design a workload where a shared codebook would be especially valuable across a network link.

Chapter 20: The Persistent Master Execution Table

The dictionary becomes more powerful when it names executable work, not just compressed bytes.

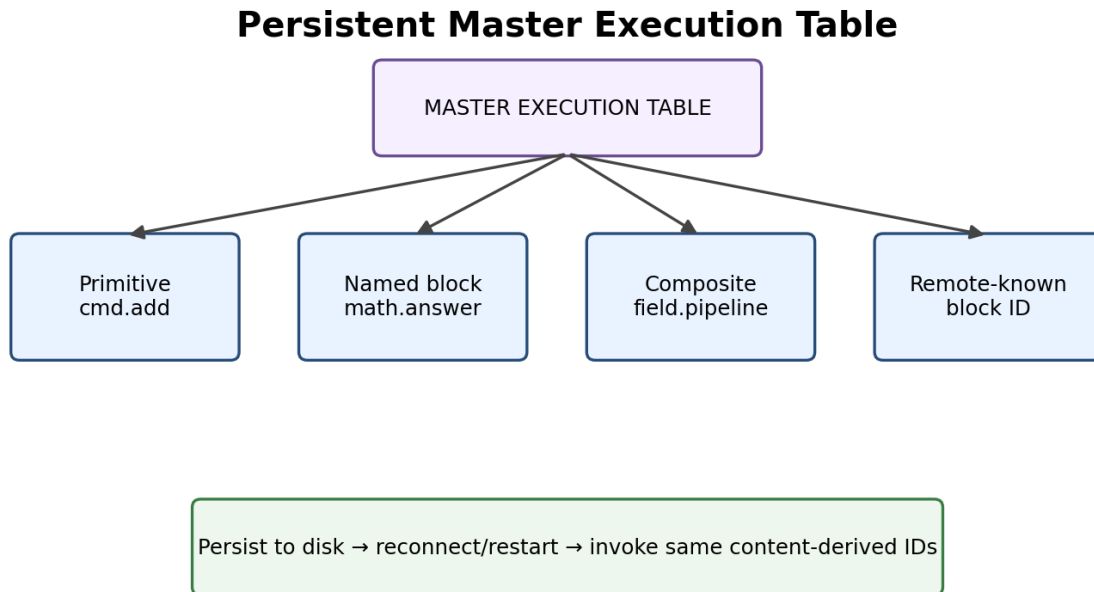


Figure 20.1 — A persistent table can turn recurring commands, blocks, and composites into a stable execution vocabulary.

The persistent master execution table assigns deterministic content IDs to primitives, bytecode blocks, and composites. Human-friendly aliases point to those identities. A remote node can install a block once, persist it, close the connection, restart, and later execute the same block by ID without receiving its code again.

This is the architectural step from “compress a program” to “build an executable vocabulary.” A composite can refer to other named blocks, so a program can become a small list of references to definitions both sides already know. Stable naming is what makes that useful: the same executable content should produce the same machine identity wherever it is installed.

Persistent shared codebooks apply the same principle to compression metadata. Once both peers know a codebook definition, later packets can carry a codebook ID plus the compact token stream. The shared-codebook benchmark in the project showed large reductions for deliberately regular workloads after the first installation, but the important programming concept is the cache lifecycle: HAS → install if missing → invoke by stable identity → keep the table across connection shutdown.

Exercises

1. Design three block aliases that all resolve to the same content identity.
2. Create a composite pipeline from three named blocks and explain what must be persisted for restart reuse.

3. Why should a codebook ID include all static decoding information, including Huffman lengths?

Part V — One Remote Computer

Treat one remote PASM executor as a persistent extension of the local machine.

Chapter 21: The Single-Peer Model

PASM's current network focus is one authenticated persistent peer rather than a general multi-node router.

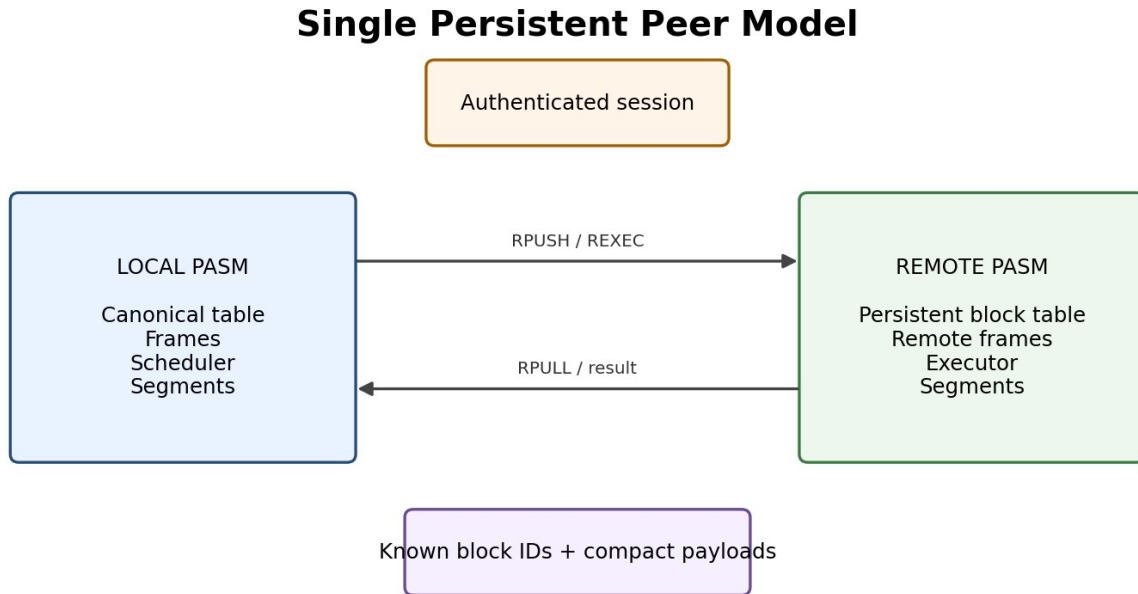


Figure 21.1 — One local controller and one remote executor share canonical block identities and operate on compact frame/register references.

The single-peer design removes destination routing from every operation. After an authenticated connection and session handshake, the local runtime can create a remote frame, push canonical register values, execute a known canonical block, pull results, and drop the frame. The connection stays open for repeated work.

The compact operations are `FRAME_NEW`, `RPUK`, `RPULL`, `REXEC`, and `FRAME_DROP`. A compact `REXEC` payload is a 4-byte frame ID plus the 12-byte binary block ID, or 16 payload bytes. `RPUK` uses 4 bytes for the frame, 2 for the canonical register ID, and 8 for the signed integer value, or 14 bytes.

The current canonical payload benchmark reports 33 bytes for a comparable JSON execute reference versus 16 bytes for the compact `REXEC` payload, a 51.5% reduction in command payload size; the register push comparison is 46 bytes versus 14, a 69.6% reduction. This benchmark is about the command envelope, not total TCP/TLS overhead.

```
$peer = (new pasm\PASMCCanonicalPeerClient('my-secret'))
->connect('tcp://127.0.0.1:9002');

$frame = $peer->frameNew('job-1');
$peer->rpush($frame, 'R0', 40);
$peer->rpush($frame, 'R1', 2);
```

```
$result = $peer->invoke($table, $block->id, $frame);  
$peer->frameDrop($frame);
```

Exercises

1. Draw the messages required to execute a known block with two integer inputs and one returned result.
2. Compare installing code every request with installing once and using REXEC by ID.
3. Explain why remote frames are recreated after reconnect while the canonical block table persists.

Chapter 22: Remote Registers and Execution Frames

Remote work is cheap when code is already known and only frame state changes.

A useful mental model is that a remote frame is a register window owned by the remote executor. RPUSH writes a canonical machine word into that window. REXEC asks the remote node to apply a known immutable block to that frame. RPULL reads a register back. This avoids serializing an arbitrary PHP object graph for every request.

```
$remoteFrame = $peer->frameNew('sum');
$peer->rpush($remoteFrame, 'R0', 100);
$peer->rpush($remoteFrame, 'R1', 23);

$execInfo = $peer->rexec($remoteFrame, $block->id);
$value = $peer->rpull($remoteFrame, 'P0');
```

The block ID identifies the algorithm; the frame ID identifies the current invocation. Multiple remote frames can run the same block with different values. This is exactly the same separation used by local scheduling, which is why the canonical model scales conceptually from one process to one remote executor.

A robust application should treat the network as fallible. A remote frame is not a durable database row. If the connection drops, the client should reconnect, negotiate the session, confirm that required block IDs are still present, create new frames, and restore whatever input state is necessary. Persist durable application data separately.

Exercises

1. Design a retry-safe remote addition request that can recreate its frame after reconnect.
2. Explain why a content-addressed block table makes reconnect cheaper.
3. Which values belong in persistent application storage instead of a remote PASM frame?

Chapter 23: Remote Segments and Dirty-Page Sync

Large mutable state should synchronize by changed page, not by retransmitting an entire container or frame.

Dirty-Page Synchronization



A canonical boundary materializes the logical state. Only pages whose encoded image changed need export.

Figure 23.1 — PASM can mirror only dirty pages when a segment’s checkpoint changes.

The segmented peer protocol adds operations for creating, dropping, reading, writing, defragmenting, inspecting, and synchronizing remote segments. The client maintains a local-segment to remote-segment mapping for each remote frame. After the first mirror, SEG_SYNC can send only dirty pages.

With a default 32-cell page and signed 64-bit cells, one full page contains 256 bytes of cell data plus a compact header. The project’s segmented peer test measured 270 bytes for a one-dirty-page sync. This makes the container checkpoint strategy important: a hot queue can perform thousands of local operations, then produce only the net changed pages at YIELD.

A particularly useful case is when a container returns to its previous canonical state before the boundary. The hot OOP layer compares the new logical image to the prior checkpoint, so there may be no dirty page to export at all. “Work happened” and “state changed at the synchronization boundary” are not the same fact.

Exercises

1. Mirror a three-page local segment, modify one value on page 2, and predict which page should be dirty.
2. Explain why a segment mapping should survive multiple sync calls within one remote frame.

3. Design a policy for flushing OOP containers before a task yields to a remote worker.

Chapter 24: Network Safety and Failure Boundaries

A remote execution system must constrain what can be installed and how long it may run.

The PASM network layers are designed around authenticated requests and constrained execution formats. Remote bytecode is interpreted by a fixed opcode VM rather than evaluating arbitrary PHP source. Canonical blocks are arrays of recognized PASM commands. The runtime can enforce program-size and instruction-count limits. These are essential safety properties for any system that receives executable work over a network.

Treat a shared secret as a development mechanism, not a complete internet-facing security architecture. Production use would require careful key management, encryption in transit, replay protection, authorization, audit logging, rate limiting, memory quotas, and a stronger process-isolation boundary. Running the remote executor under a restricted operating-system account or container is a sensible defense-in-depth step.

DO NOT EXPOSE BLINDLY: Do not bind an experimental remote execution service to the public internet without a complete security review. A whitelist-based PASM VM is safer than arbitrary PHP execution, but networking, parsers, resource exhaustion, and implementation bugs remain attack surfaces.

Make failure explicit in application design. Timeouts, disconnects, unknown block IDs, stale codebook versions, invalid frames, instruction-limit exceptions, divide-by-zero, invalid jumps, and segment bounds errors should all be treated as ordinary failure modes rather than impossible conditions.

Exercises

1. List five limits you would enforce on an internet-facing PASM executor.
2. Explain why “no arbitrary PHP callbacks” is an important remote boundary.
3. Design a reconnect sequence that verifies block availability before reissuing work.

Part VI — Projects and Performance

Put the layers together, then measure the system you actually built.

Chapter 25: Project: A Reusable Remote Calculator

Build a small service where the code is installed once and only inputs/results move on later calls.

The simplest practical remote PASM pattern is a library of canonical arithmetic blocks. The local controller defines the blocks and ensures they exist remotely. For each request it creates a remote frame, writes inputs, executes a known block ID, reads P0, and drops the frame. The remote node persists the block table, so the algorithm does not need to be retransmitted after restart.

```
// Define once locally.
$add = $stable->define('calc.add', [
    ['ADD', 'P0', 'R0', 'R1'],
    ['RET', 'P0'],
]);

// Per request.
$f = $peer->frameNew('calc-request');
$peer->rpush($f, 'R0', 1000);
$peer->rpush($f, 'R1', 24);
$r = $peer->invoke($stable, $add->id, $f);
$peer->frameDrop($f);
```

Expand the service by defining subtraction, multiplication, integer division, modulo, bitwise operations, and small composite blocks. Keep each canonical block bounded and deterministic. If a computation becomes large or repetitive, compile it to bytecode or use a fused canonical block instead of turning every arithmetic step into a separate network request.

The calculator illustrates the PASM network principle: maximize work per remote invocation and minimize protocol chatter. The strongest reason to send execution to the peer is not that a remote CPU is magically faster; it is that the peer may already hold the data, codebook, block table, or specialized state needed to do the work locally.

Exercises

1. Add a `calc.mul` block and test it with two remote frames concurrently.
2. Create a safe division block and decide how your application will represent divide-by-zero failure.
3. Measure total wire bytes for ten known-block requests versus reinstalling the code every time.

Chapter 26: Project: Cooperative Work Pipeline

Combine named blocks, frames, containers, and YIELD to build an inspectable multi-stage task.

Suppose each job passes through receive, transform, aggregate, and emit stages. Put job-specific scalar state in a frame, mixed collections in OOP containers attached to that frame, and larger integer working sets in segments. Make each stage a named block or PHP-level orchestration step and synchronize only at the boundaries that matter.

```
// Pseudocode / orchestration pattern
$task = $scheduler->spawn('pipeline.block', [
    'R0' => $jobId,
], 'job-' . $jobId);

// Inside the block or orchestration:
// stage 1 -> YIELD
// stage 2 -> YIELD
// stage 3 -> RET
```

A good pipeline design has explicit ownership. The frame owns scalar execution state. A queue or map object belongs to the frame. Segment handles live in persistent registers such as P1/P2. The scheduler owns the program counter. The canonical table owns code. The remote peer owns only the remote copies of state that have actually been synchronized.

This separation makes performance tuning easier because every layer has a clear boundary. If a container is slow, optimize its hot representation. If remote traffic is large, inspect dirty pages and block reuse. If the interpreter is hot, inspect fusion opportunities. If defrag is frequent, inspect allocation patterns.

Exercises

1. Design a three-stage task and identify every YIELD point.
2. Choose which state goes in registers, a Queue, a Map, and a segment.
3. Add a remote stage and state exactly what must be synchronized before REXEC.

Chapter 27: Project: Segmented Lookup Processor

Use segments when an algorithm repeatedly scans or updates page-oriented integer state.

A lookup processor is a good fit for PASM segments because handles are stable and pages can be synchronized independently. Allocate the table once in a frame, store the segment ID in P1, and run bounded blocks that read or update offsets. If allocation/deallocation produced fragmentation, compact once before a segmented hot loop rather than repeatedly inside it.

```
$block = $table->define('lookup.sum-two', [  
    ['SEGGET', 'R0', 'P1', 0],  
    ['SEGGET', 'R1', 'P1', 33],  
    ['ADD', 'P0', 'R0', 'R1'],  
    ['RET', 'P0'],  
]);
```

If the lookup table is mirrored to the remote peer, keep the mapping alive and send dirty pages after local edits. If the remote peer is the primary executor, consider keeping the authoritative working copy remote and pulling only final results rather than ping-ponging the segment on every phase.

Exercises

1. Build a 96-cell table, place values on three different pages, and sum them.
2. Free an unrelated segment, defrag, and prove the lookup segment handle still works.
3. Compare two designs: local-authoritative with dirty-page sync versus remote-authoritative with result-only pulls.

Chapter 28: Benchmarking PASM Correctly

Performance engineering requires workload definitions, repeatability, and honest comparison boundaries.

A PASM benchmark should identify exactly what is being timed: PHP object construction, container API calls, canonical checkpoint flush, bytecode compilation, precompiled VM execution, compression, network payload size, or end-to-end remote latency. Mixing these into one number makes the result difficult to interpret.

For local microbenchmarks, use a high-resolution clock such as `hrtime(true)`, warm the code path, repeat cases, and report medians. Enable CLI OPcache consistently if you use it. Keep the same PHP version, CPU power mode, iteration count, and input size when comparing old and new PASM versions.

```
$start = hrtime(true);
for ($i = 0; $i < $n; $i++) {
    $stack->push($i);
}
$ms = (hrtime(true) - $start) / 1e6;
```

For network work, report payload bytes separately from end-to-end latency. A 16-byte REXEC payload does not mean a 16-byte TCP transaction; framing, authentication, transport headers, acknowledgements, and the response still exist. Payload minimization is still valuable because it reduces serialization work and makes repeated protocols more compact, but describe what you measured.

For compression, always test both compressible and irregular data. PASM's codebook benchmark intentionally repeats the same five-instruction block, so its strongest ratios demonstrate the mechanism under high regularity rather than typical arbitrary code.

Exercises

1. Write a benchmark that measures only Vector add/get, then a second benchmark that includes flush().
2. Explain why compile+execute and precompiled-execute should be reported separately.
3. List three sources of noise in a microbenchmark on a desktop computer.

Chapter 29: Current Benchmark Results

The current build improves PASM's legacy OOP layer while still trailing native PHP arrays on basic container operations.

1,000,000 API ops	Legacy ms	Canonical hot-path ms	PASM speedup	Native PHP ms
Vector add/get	53.924	42.449	1.27×	8.189
Stack push/pop	80.414	46.253	1.74×	14.645
Queue enq/deq	88.356	67.362	1.31×	8.815
Deque back/front	96.006	83.465	1.15×	9.306
Map put/get	48.917	45.220	1.08×	9.232
Set add/has	240.272	152.258	1.58×	10.117

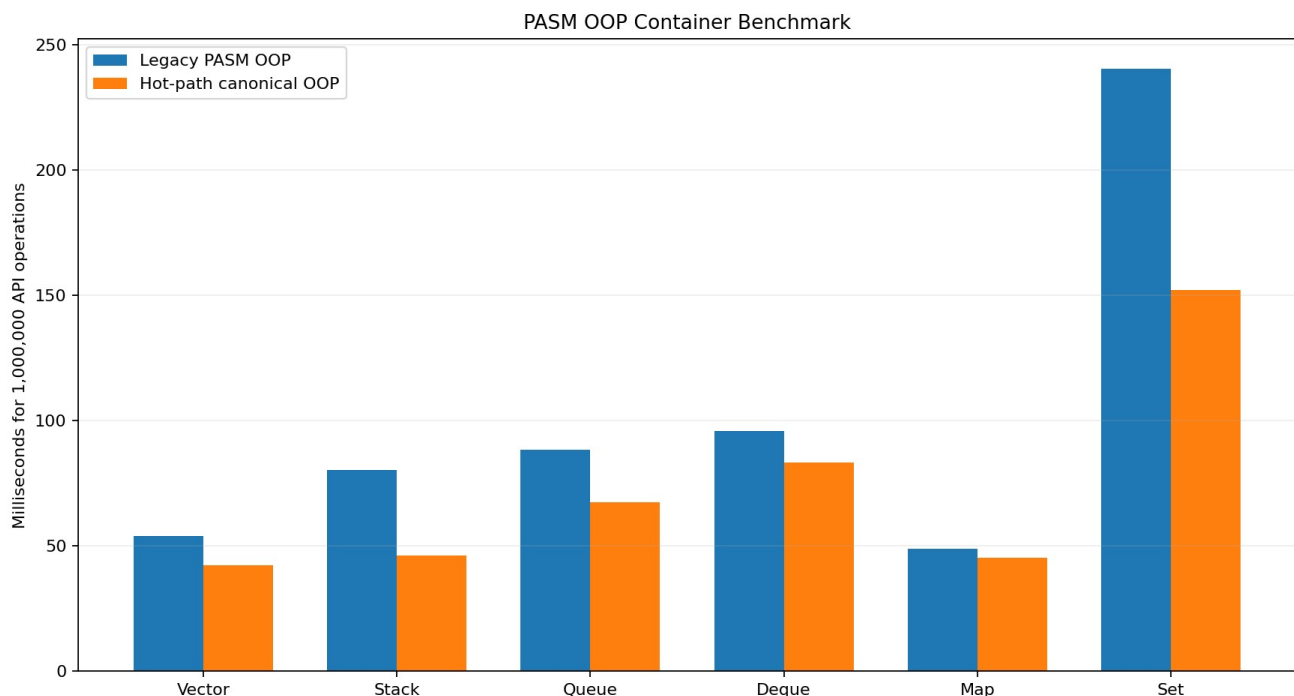


Figure 29.1 — Legacy versus current PASM OOP timings for one million API operations. Native PHP remains faster on these microbenchmarks.

The table shows the right engineering interpretation. The hot-path rewrite is faster than the legacy PASM OOP layer in all six recorded container workloads, and memory peak fell from about 56 MB to about 52 MB in the one-million-operation run. However, native PHP arrays remain substantially faster for these basic operations. PASM is paying for object APIs, frame ownership, checkpoint capability, and runtime integration. The goal is to keep that premium bounded and obtain value from the additional machine semantics.

Other measurements rerun for this manual include:

Area	Current measured result
------	-------------------------

Canonical frame microbenchmark	100,000 create/set/invoke/drop runs: ~582.549 ms (~171,659 runs/s).
Canonical payload	REXEC 16 B vs 33 B JSON-style reference; RPUSH 14 B vs 46 B.
Segmentation	Defrag ~0.834 ms for the test fixture; 100k write/read pairs ~37.666 ms.
Bytecode	10k precompiled four-instruction executions ~10.585 ms; 26-byte base program.
Optimizer	20k 25-step loops: ~333.339 ms base vs ~200.541 ms optimized (1.66×).
Codebook	500 repeated blocks: 13,502 B raw → 670 B wire on deliberately repetitive input.

INTERPRETATION: PASM has measured internal improvements and compact protocol representations. It has not established that it universally “defeats WebAssembly.” A serious WASM comparison must run equivalent workloads, engines, data movement, startup, and network boundaries on the same hardware.

Exercises

1. Choose one benchmark above and rerun it five times on your machine. Report median and range.
2. Add flush() to the OOP benchmark and quantify checkpoint cost separately.
3. Design a fair PASM-versus-WASM benchmark for a network-native repeated block rather than a raw integer-add loop.

Chapter 30: Performance Tuning Checklist

Optimize the layer that is actually hot, and preserve PASM's separation of concerns.

When a PASM workload is slow, start with measurement and then inspect the relevant layer. Container hot paths should stay local. Segmentation should normally happen at boundaries. Defragmentation should happen when fragmentation justifies it, not blindly. Remote code should be installed once and executed by ID. Repeated assembly should be compiled once. Hot bytecode patterns can be fused. Repeated program structures can use a shared codebook. None of these optimizations should be enabled merely because they exist.

- Container hot loop slow? Avoid flush/defrag/register bridges inside the loop; choose the correct container type.
- Deque slow? Confirm you are using the circular implementation, not a legacy array-rebuilding path.
- Set slow? Check whether values force the complex serialization fallback rather than typed scalar signatures.
- Segment access slow? Batch work by page and avoid unnecessary read/write pairs across abstraction layers.
- Remote traffic large? Confirm blocks/codebooks are cached and only dirty pages or result registers move.
- Interpreter dispatch hot? Profile actual traces and use vetted superinstructions.
- Compression costly? Let raw fallback win on small or irregular programs.
- Too many defrags? Track fragmentation statistics and compact once before a segmented hot phase.

The architecture deliberately makes these costs visible. A monolithic runtime can hide why work is expensive; PASM's explicit layers let you ask whether the cost is computation, dispatch, canonicalization, memory movement, compression, or networking.

Exercises

1. Take one of your PASM programs and label every operation as compute, container, segment, checkpoint, or network work.
2. Move one unnecessary boundary operation out of a hot loop and remeasure.
3. Write a rule for when your application should choose raw bytecode instead of a codebook packet.

Chapter 31: What to Build Next

The strongest next work is not “more features”; it is tighter unification, correctness tests, and fair comparative benchmarks.

PASM now has enough layers that adding another abstraction without consolidation could make the system harder to reason about. The next engineering stage should focus on one canonical programming surface that can compile to the local executor, bytecode VM, or remote peer while preserving the same register/frame/segment semantics. The OOP checkpoint layer should integrate directly with the segmented peer so dirty pages and pooled mixed values travel as one coherent snapshot format.

A second priority is generated instruction/reference documentation from the source. The manual includes a hand-curated reference, but the runtime can generate machine-readable opcode tables, register contracts, and container APIs. That would make compatibility testing between releases easier.

A third priority is a rigorous WASM comparison. A fair test should include at least: cold startup, repeated warm local arithmetic, object/container workloads, serialization/host boundary cost, one persistent remote peer, repeated known-code remote execution, and large regular code transfer with a pre-shared dictionary. PASM should be allowed to win only where the measurement shows it wins.

Finally, consider a stable versioned ISA. Canonical IDs are most useful when semantics do not silently change. A block identity should include or be scoped by the ISA version so two peers never treat the same content bytes under incompatible semantics.

Exercises

1. Design an ISA version field that should participate in remote handshake and block validation.
2. List the tests needed before calling the current canonical OOP layer “stable.”
3. Write a benchmark matrix with at least six workloads for a PASM-versus-WASM study.

Part VII — Exercises and Guided Practice

Use these assignments to turn the architecture into working skill.

Capstone Exercises

Capstone 1 — Canonical arithmetic library

Create add, subtract, multiply, modulo, and bounded loop blocks. Give each a schema. Run them in multiple isolated frames and write a test that verifies no cross-talk.

Capstone 2 — Cooperative queue processor

Use a PASM Queue plus a canonical frame. Process jobs in batches, yield between batches, and flush the queue only at the yield boundary. Measure hot-work time and checkpoint time separately.

Capstone 3 — Segmented lookup table

Allocate at least four pages, create deliberate fragmentation, defrag, then run a loop that reads multiple offsets. Verify logical handles and data remain stable.

Capstone 4 — Optimized bytecode

Write a small loop in PASM assembly, compile it, benchmark base and optimized bytecode, and inspect which superinstruction pattern was selected.

Capstone 5 — Best-bet compression

Create a repetitive and an irregular program of similar raw size. Compare raw/codebook choices, prediction rate, and wire size. Explain why the transport chooses differently.

Capstone 6 — Persistent remote service

Run a canonical node locally. Install a block, invoke it, restart the node, reconnect, verify the block remains known, and invoke it again using a fresh frame.

Capstone 7 — Dirty-page remote state

Mirror a multi-page segment, modify values in one page only, sync, and confirm only that page is dirty. Repeat with an edit that returns the page to its prior state before the boundary.

Capstone 8 — Fair comparison

Design and implement one PASM benchmark and one equivalent implementation in another runtime. State exactly what each measurement includes and excludes.

Selected Solution Hints

Canonical arithmetic

Use one immutable block per operation and pass operands in R0/R1. Return P0 or R2 consistently. The test should create new frames rather than mutate one frame repeatedly when proving isolation.

Queue processor

Keep enqueue/dequeue on the hot local representation. Call `dirtySegments()` or `flush()` after the batch, not after each item.

Segmented lookup

Store the segment handle in P1. Record `descriptor(start)` before and after defrag. Assert that reads by the same P1 return the same values.

Bytecode optimizer

Benchmark precompiled execution separately. A loop with DEC/CMP/JNZ is a good candidate for DEC_CMP_JNZ fusion.

Codebook

The repetitive program should eventually use codebook mode; a tiny or irregular program may remain raw. That is correct behavior.

Remote restart

Persist the canonical table file on the node. Frames are not expected to survive restart; create a new one.

Appendices

Reference material for day-to-day PASM programming.

Appendix A — Canonical Instruction Reference

Instruction	Form	Meaning
MOVI	MOVI dst, imm	Write an immediate integer.
MOV	MOV dst, src	Copy a canonical register.
ADD	ADD dst, a, b	Integer addition.
SUB	SUB dst, a, b	Integer subtraction.
MUL	MUL dst, a, b	Integer multiplication.
DIV	DIV dst, a, b	Integer division; b must be nonzero.
MOD	MOD dst, a, b	Integer modulo; b must be nonzero.
AND / OR / XOR	OP dst, a, b	Bitwise operations.
INC / DEC	INC reg / DEC reg	Increment or decrement one register.
CMP	CMP a, b	Set equality/zero comparison state.
JMP	JMP pc	Unconditional command-index jump.
JZ / JNZ	JZ pc / JNZ pc	Conditional jump based on CMP equality.
RET	RET value	Return a register/immediate and copy result to P0.
YIELD	YIELD	Return control while preserving frame + PC.
HALT	HALT	Stop without requiring a result.

Segmented commands

Instruction	Purpose
SEGNEW / PAGE_NEW	Allocate a page-aligned segment and store the logical ID in a destination register.
SEGFREE / PAGE_FREE	Free a logical segment.
SEGSET / PAGE_SET	Write one integer cell.
SEGGET / PAGE_GET	Read one integer cell into a register.
SEGFILL	Fill a range.
SEGCOPY	Copy cells between segments.
DEFRAG / SEGDEFRAG / SEGPack / PACKPAGES	Compact live segments toward page zero.
SEGSTAT	Read segment-arena metrics.

Compact peer operations

Operation	Purpose	Compact payload concept
FRAME_NEW	Create remote execution frame	name/session request
RPUSH	Write remote canonical register	frame(4)+reg(2)+value(8)
RPULL	Read remote canonical register	frame+reg
REXEC	Execute known canonical block	frame(4)+block id(12)
FRAME_DROP	Release remote frame	frame id

Appendix B — OOP Container API

Container	Primary methods
Vector / PASMList	add, insert, get, set, removeAt, first, last, contains, ArrayAccess
Stack / PASMStack	push, pop, peek + list methods
Queue / PASMQueue	enqueue, dequeue, peek
Deque / PASMDeque	pushBack, pushFront, popFront, popBack, peekFront, peekBack
Map / PASMMap	put, get, has, remove, keys, values, ArrayAccess
Set / PASMSet	add, has, remove, union, intersect, difference

Shared canonical container methods

Method	Use
frame()	Return owning PASMRegisterFrame.
containerId()	Frame-local container identity.
segmentIds()	Logical checkpoint segment IDs.
dirtySegments()	Flush if needed and return dirty page map.
clearDirty()	Mark current checkpoint clean.
flush()	Materialize the current logical state into segmented checkpoint storage.
defrag()	Flush, then compact the frame arena.
loadRegister(P0)	Store canonical container ID in a frame register; legacy names bridge to old PASM.
fromRegister(P0)	Replace contents from a referenced container or compatible legacy value.
loadPasmStack() / fromPasmStack()	Explicit bridge to legacy PASM stack.

Appendix C — Data Structure Reference

Class	Operations
SinglyLinkedList	append, prepend, shift, first, last, contains, removeFirst
DoublyLinkedList	pushFront, pushBack, popFront, popBack
BinarySearchTree	put, has, get, min, max
BinaryHeap / PASMHeap	minHeap, maxHeap, push, peek, pop
Graph	addVertex, addEdge, neighbors, bfs, dfs
Trie	put, has, get, prefix
HashTable	put, get, has, remove
DisjointSet	makeSet, find, union, connected

Appendix D — Benchmark Reproduction

Run these commands from the directory containing the current PASM files:

```
php -d opcache.enable_cli=1 benchmark-pasm-oop-fast.php
php -d opcache.enable_cli=1 benchmark-pasm-canonical.php
php -d opcache.enable_cli=1 benchmark-pasm-segmentation.php
php -d opcache.enable_cli=1 benchmark-pasm-bytecode.php 10000
php -d opcache.enable_cli=1 benchmark-pasm-optimizer.php 20000
php -d opcache.enable_cli=1 benchmark-pasm-codebook.php
```

Record the exact PHP version, CPU, operating system, CLI OPcache setting, and iteration counts with your results. If you change PASM code, compare against the previous build on the same environment.

Appendix E — Glossary

Term	Definition
Canonical block	Immutable command array plus schema whose ID is derived from content.
Frame	Mutable register state for one execution instance.
R/T/P/M banks	General, temporary, persistent/result, and machine registers.
Segment	Stable logical handle to page-aligned integer storage owned by a frame.
Page	Fixed-size allocation/synchronization unit, default 32 cells in the segmented runtime.
Defragmentation	Compaction of PASM logical segment placement; not PHP heap compaction.

Checkpoint	Canonical segmented image of a hot OOP container at a synchronization boundary.
Dirty page	A page whose canonical image differs from the previously clean checkpoint.
Superinstruction	One interpreter opcode representing a safe recurring multi-instruction pattern.
Adaptive profile	Observed execution-trace counts used to select registered optimizations.
Best bet	Most frequent next token for a context in the deterministic predictor.
Codebook	Dictionary + prediction/Huffman information used to encode recurring bytecode.
Master table	Persistent lookup table of named/content-addressed executable work.
Single peer	One persistent authenticated remote PASM executor connection.
REXEC	Execute a known canonical block against a remote frame.
Raw fallback	Decision to send uncompressed bytecode when compression would be larger.

Appendix F — Troubleshooting

Unknown canonical register

Use R0-R255, T0-T1023, P0-P255, or M0-M7. Check spelling and range.

Unknown canonical block

Ensure the table contains the ID/alias. On a peer, call ensureBlock/install before REXEC.

Container register reference fails

The canonical register must contain a container ID that exists in the same frame.

Unexpected container sync cost

Move dirtySegments/flush/defrag out of the hot loop and into a boundary.

Deque unexpectedly slow

Verify the hot-path circular implementation is loaded, not the legacy container file.

Segment bounds error

Check logical length and offset; page padding does not increase the legal logical length.

Remote frame missing after reconnect

Expected behavior. Recreate session frames and restore inputs; persistent block table is separate.

Compression larger than raw

Expected on small/irregular programs. Let raw fallback win.

Optimizer changes result

Treat as a correctness bug. Disable the fusion and add a regression test before using it again.

Benchmark changed dramatically

Check PHP version, OPcache, background load, iteration count, and whether boundary costs were included.

Appendix G — Migration from Legacy PASM

A practical migration keeps legacy PASM working while moving stateful subsystems one at a time. Start by identifying global static-register assumptions. Convert a reusable algorithm into a canonical block and map its inputs/outputs to R/P registers. Then introduce separate frames for concurrent invocations. Move large integer state into frame-owned segments. Replace high-traffic legacy collections with the hot-path OOP containers. Finally, add peer execution only after local canonical behavior is covered by tests.

1. Choose one legacy function and write a local canonical equivalent.
2. Add a regression test proving both versions return the same result for representative inputs.
3. Move the canonical version into two independent frames and test isolation.
4. If the algorithm uses collections, move them to the OOP layer and flush only at explicit boundaries.
5. If the algorithm is repeated, compile or cache the block rather than retransmitting source.
6. Only then expose it through the remote peer, with instruction/resource limits and reconnect handling.

Appendix H — Design Invariants

- Same executable content + same schema should have the same canonical block identity within an ISA version.
- Block identity describes code; frame identity describes one execution instance.
- One frame's registers and segments must not leak into another frame.
- Segment logical IDs remain valid across PASM defragmentation.
- OOP hot operations stay off the segmented codec path until an explicit boundary.
- Remote execution never requires arbitrary PHP source evaluation.
- Optimizations are optional and must preserve observable semantics.
- Compression may decline to compress when raw bytecode is smaller.

- Best-bet prediction is deterministic statistics, not AI.
- Benchmarks describe a specific workload and environment; they are not universal claims.

Closing Note

PASM is most coherent when it is programmed as a machine with explicit boundaries. Blocks are vocabulary. Frames are instances. Registers are machine words. Containers are fast local objects with checkpoint images. Segments are stable logical storage. Yield is a scheduling boundary. Codebooks are reusable compression knowledge. The peer is one remote extension of the same canonical identity system. Keeping those roles separate is what allows the runtime to become more capable without forcing every operation to pay for every feature.